



TẬP ĐOÀN CÔNG NGHIỆP - VIỄN THÔNG QUÂN ĐỘI

BÁO CÁO MINI-PROJECT

**XÂY DỰNG SELF-EVOLVING AI AGENTS
TRONG XỬ LÝ SỰ CỐ MẠNG VIỄN THÔNG**

Nguyễn Thanh Hải

optimus1072005@gmail.com

Chương trình Viettel Digital Talent 2026

Lĩnh vực: Data Science & AI Engineering

Mentor: Nguyễn Duy Hưng

Đơn vị: Tổng công ty Mạng lưới Viettel

HÀ NỘI - 2026

Tóm tắt

Sự phát triển nhanh chóng của các hệ thống mạng viễn thông hiện đại đã đặt ra nhiều thách thức lớn trong quá trình vận hành và xử lý sự cố. Các phương pháp chẩn đoán truyền thống thường dựa vào luật thủ công hoặc các mô hình AI tĩnh, gặp nhiều khó khăn trong việc thích ứng với các lỗi mới hoặc sự thay đổi của cấu trúc mạng. Nhằm giải quyết vấn đề này, đề tài tập trung nghiên cứu, xây dựng và đánh giá hệ thống ****Self-Evolving AI Agents**** hỗ trợ tự động xử lý sự cố mạng dựa trên nền tảng ****NIKA**** (A Network Arena for Benchmarking AI Agents on Network Troubleshooting).

Nền tảng NIKA cung cấp môi trường mô phỏng mạng thực tế dựa trên Kathará, hỗ trợ giả lập nhiều sự cố phức tạp (như lỗi cấu hình định tuyến BGP/OSPF, lỗi thiết bị mạng, SDN controller và P4 data-plane) cùng cơ chế đánh giá hiệu năng chẩn đoán tự động. Nghiên cứu này hiện thực hóa hai mô-đun cốt lõi giúp AI Agent có khả năng tự tiến hóa:

1. ****Mô-đun Bộ Nhớ Quy Trình Tiến Hóa (Composable Procedural Memory Module):****
Sử dụng lưu trữ kết hợp PostgreSQL và Qdrant để lưu trữ kinh nghiệm chẩn đoán dưới dạng các ghi chú nguyên tử (atomic notes). Mô-đun sử dụng cơ chế nén vết chẩn đoán (trajectory compaction) lấy cảm hứng từ LightMem, truy xuất theo thuộc tính ngữ cảnh (attribute-aware retrieval) lấy cảm hứng từ MemInsight, và cập nhật đồ thị liên kết lấy cảm hứng từ A-Mem. Quá trình tiến hóa bộ nhớ được kiểm soát nghiêm ngặt bằng điểm số đánh giá từ Evaluator (score-gated validation) nhằm tránh rò rỉ thông tin đáp án của benchmark.
2. ****Mô-đun Tiến Hóa Công Cụ (Tool Evolution Module):**** Hỗ trợ AI Agent tự phát triển năng lực sử dụng công cụ tại test-time (lấy cảm hứng từ DRAFT, TTE và Alita). Agent có thể tự động Master mô tả các công cụ nguyên thủy (primitive tools), tổng hợp thành các quy trình chẩn đoán phức tạp (composite workflows), hoặc sinh mã nguồn Python (generated Python helpers) để xử lý dữ liệu. Toàn bộ các công cụ mới đều được kiểm tra cú pháp AST và chạy kiểm định trong môi

trường Docker sandbox an toàn trước khi lưu trữ lâu dài.

Hiệu quả của hệ thống được kiểm chứng thông qua các thực nghiệm chẩn đoán chi tiết trên tập con 30 sự cố chọn lọc từ benchmark NIKA. Kết quả cho thấy agent được trang bị mô-đun tiến hóa đạt độ chính xác chẩn đoán vượt trội so với baseline thông thường (điểm F1 chẩn đoán tăng từ 0.0111 lên 0.1611, cải thiện đáng kể khả năng chẩn đoán trong các sự cố mất cấu hình định tuyến và IP), đồng thời giảm số lượng bước gọi công cụ thừa và tối ưu hóa chi phí token sử dụng.

Từ khóa: *Self-Evolving AI Agents, xử lý sự cố mạng, NIKA, bộ nhớ quy trình tiến hóa, tiến hóa công cụ, Kathará.*

Lời cảm ơn

Em xin gửi lời cảm ơn chân thành và sâu sắc nhất tới người đã hướng dẫn và đồng hành cùng em trong dự án, anh **Nguyễn Duy Hưng**, chuyên viên Khoa học Dữ liệu, Trung tâm Chuyển đổi số, Tổng công ty Mạng lưới Viettel, anh **Bùi Quang Hưng** ... Trong suốt thời gian thực hiện dự án, sự hỗ trợ và động viên của anh không chỉ thúc đẩy và giúp đỡ em về mặt chuyên môn mà còn định hướng cho sự phát triển của em trên con đường theo đuổi sự nghiệp sau này.

Đồng thời, em cũng xin gửi lời cảm ơn tới tất cả giảng viên đào tạo lĩnh vực **Data Science & AI Engineer** tại chương trình **Viettel Digital Talent 2026**. Kiến thức và những chia sẻ từ các anh chị đã và sẽ giúp đỡ chúng em rất nhiều trong quá trình học tập, làm việc và phát triển trong tương lai. Bên cạnh đó, em cũng xin gửi lời cảm ơn tới tất cả các bạn thực tập sinh trong lĩnh vực Data Science & AI Engineer nói riêng và toàn chương trình nói chung vì đã giúp đỡ và đồng hành cùng nhau trong quá trình học tập tại chương trình năm nay.

Cuối cùng, em xin cảm ơn **Tập đoàn Công nghiệp - Viễn thông Quân đội Viettel** vì đã cho em cơ hội tham gia và học tập tại chương trình VDT 2026, một hành trình với không chỉ kiến thức mà còn là những kinh nghiệm và cơ hội mới, là hành trang cho hành trình theo đuổi tương lai.

Lời cam đoan

Tôi, Nguyễn Thanh Hải, sinh viên trường Đại học Công nghệ - Đại học Quốc gia Hà Nội, thực tập sinh lĩnh vực đào tạo Data Science & AI Engineer, chương trình Viettel Digital Talent 2026, xin cam đoan dự án với đề tài “Xây dựng Self-Evolving AI Agents trong xử lý sự cố mạng viễn thông” là sản phẩm nghiên cứu gốc của tôi. Tôi xin cam đoan toàn bộ nội dung trình bày trong báo cáo hoàn toàn không sao chép từ bất kỳ tài liệu nào khác. Ngoài ra, tôi cũng không sử dụng bất kỳ một phát biểu, kết quả hay công trình nghiên cứu của người khác mà không trích dẫn đầy đủ. Tôi xin cam mã nguồn của dự án này là do tôi tự phát triển, không sao chép mã nguồn của người khác. Tôi xin chịu hoàn toàn trách nhiệm theo quy định đối với các cam kết của mình.

Thực tập sinh

Nguyễn Thanh Hải

Mục lục

Tóm tắt	iii
Lời cảm ơn	v
Lời cam đoan	vi
Mục lục	vii
Danh mục hình ảnh	viii
1 Giới thiệu chung	1
1.1 Đặt vấn đề	1
1.2 Phát biểu bài toán	2
1.3 Đóng góp của dự án	3
2 Kiến thức nền tảng	5
2.1 Mạng viễn thông	5
2.2 Phân tích nguyên nhân gốc rễ	6
2.3 Mô hình ngôn ngữ lớn	7
2.4 AI Agent	7
2.5 Self-Evolving AI Agents	8
2.5.1 Các thành phần có thể tiến hóa	9
2.5.2 Thời điểm tiến hóa	10
2.5.3 Các phương thức tiến hóa thường gặp	11
2.5.4 Các công trình nghiên cứu tự tiến hóa nền tảng	14
2.5.5 Vai trò trong xử lý sự cố mạng viễn thông	15
3 Kiến trúc hệ thống	17
3.1 Kiến trúc tổng quan	17
3.2 Môi trường mạng ảo hóa và các dịch vụ MCP	18
3.3 Các luồng suy luận chẩn đoán của AI Agent	18

3.4	Mô-đun Bộ Nhớ Quy Trình Tiến Hóa (Memory Module)	19
3.4.1	Cơ chế lưu trữ hỗn hợp	19
3.4.2	Truy xuất nhận biết thuộc tính (MemInsight-style Retrieval) . . .	19
3.4.3	Nén vết tương tác (LightMem-style Compaction)	20
3.4.4	Đồ thị liên kết và Cơ chế lọc điểm số (A-Mem & Score-gated Validation)	20
3.5	Mô-đun Tiến Hóa Công Cụ (Tool Evolution Module)	21
3.5.1	Mastery mô tả công cụ (DRAFT-like Refinement)	21
3.5.2	Tổng hợp quy trình phức hợp và sinh mã nguồn Python (TTE & Alita-style)	21
3.5.3	Quy trình xác thực an toàn (AST & Sandbox Validation)	21
4	Thực nghiệm	23
4.1	Thiết lập thực nghiệm	23
4.2	Bộ kịch bản sự cố và cơ chế chia dữ liệu	24
4.3	Kết quả thực nghiệm	25
4.4	Phân tích chi tiết và thảo luận	26
4.4.1	Các sự cố có sự cải thiện lớn nhất (Top Gains)	26
4.4.2	Các sự cố bị suy giảm hiệu năng (Top Losses)	27
5	Kết luận và hướng phát triển	28
5.1	Kết quả đạt được của dự án	28
5.2	Những hạn chế hiện tại	29
5.3	Định hướng phát triển tương lai	29

Danh sách hình vẽ

2.1	Bức tranh phát triển của một số framework tiêu biểu trong lĩnh vực Self-Evolving AI Agents giai đoạn 2022–2025. Hình được tổ chức theo trình tự thời gian, thể hiện các mốc nghiên cứu quan trọng trong quá trình phát triển agent có khả năng lập kế hoạch tự động, sử dụng công cụ và tự cải thiện liên tục.	9
-----	--	---

Chương 1

Giới thiệu chung

Chương này trình bày góc nhìn tổng quan về đề tài nghiên cứu, bao gồm ba nội dung chính. Đầu tiên, phần 1.1 đưa ra bối cảnh, động lực và lý do lựa chọn đề tài. Tiếp theo, phần 1.2 phát biểu cụ thể bài toán cần giải quyết trong dự án. Cuối cùng, phần 1.3 trình bày những đóng góp chính của dự án trong việc nghiên cứu và xây dựng hệ thống Self-Evolving AI Agents hỗ trợ xử lý sự cố mạng viễn thông.

1.1 Đặt vấn đề

Sự phát triển nhanh chóng của các hệ thống mạng viễn thông hiện đại đã đặt ra nhiều thách thức lớn trong quá trình vận hành, giám sát và xử lý sự cố. Các mạng thế hệ mới, đặc biệt là mạng di động 5G, có quy mô ngày càng lớn, cấu trúc ngày càng phức tạp và bao gồm nhiều thành phần có quan hệ phụ thuộc chặt chẽ với nhau như trạm gốc, thiết bị vô tuyến, lõi mạng, cấu hình truyền dẫn, tham số handover và các chỉ số hiệu năng mạng. Trong môi trường đó, sự cố có thể xuất hiện từ nhiều nguyên nhân khác nhau, từ lỗi phần cứng, sai cấu hình, nhiễu tín hiệu, suy giảm vùng phủ cho đến quá tải tài nguyên. Nếu không được phát hiện và xử lý kịp thời, các sự cố này có thể làm suy giảm chất lượng dịch vụ, gây gián đoạn kết nối, ảnh hưởng trực tiếp đến trải nghiệm người dùng và làm tăng chi phí vận hành của nhà mạng.

Trong thực tế, xử lý sự cố mạng viễn thông không chỉ dừng lại ở việc phát hiện dấu hiệu bất thường, mà còn yêu cầu xác định nguyên nhân gốc rễ đứng sau các hiện tượng quan sát được. Đây là bài toán khó vì dữ liệu vận hành mạng thường có kích thước lớn, nhiều chiều, thay đổi liên tục theo thời gian và chứa các mối quan hệ nhân quả phức tạp giữa nhiều thành phần. Các phương pháp truyền thống thường dựa vào luật thủ công, kinh nghiệm của chuyên gia hoặc các mô hình học máy tĩnh. Tuy nhiên, những phương

pháp này gặp hạn chế khi hệ thống mạng thay đổi, khi xuất hiện các dạng lỗi mới hoặc khi cần đưa ra lời giải thích rõ ràng cho quá trình chẩn đoán.

Trong những năm gần đây, các mô hình ngôn ngữ lớn và AI Agent mở ra nhiều tiềm năng mới cho bài toán xử lý sự cố nhờ khả năng phân tích dữ liệu, suy luận theo ngữ cảnh, sử dụng công cụ và sinh giải thích bằng ngôn ngữ tự nhiên. Tuy vậy, các agent thông thường vẫn mang tính tĩnh, tức là hoạt động chủ yếu dựa trên tri thức, prompt, bộ nhớ hoặc chiến lược xử lý được thiết kế sẵn. Điều này khiến chúng khó thích nghi lâu dài với môi trường mạng luôn thay đổi. Từ đó, nhu cầu xây dựng các **Self-Evolving AI Agents** trở nên cần thiết. Đây là các agent có khả năng tự cải thiện dựa trên dữ liệu mới, phản hồi từ người dùng, kết quả xử lý trong quá khứ và kinh nghiệm tích lũy trong quá trình tương tác.

Chính nhu cầu đó đã trở thành động lực chính cho nghiên cứu này. Trong dự án này, chúng tôi tập trung nghiên cứu và xây dựng một hệ thống AI Agent có khả năng tự cải thiện nhằm hỗ trợ phân tích và xử lý sự cố mạng viễn thông. Hệ thống hướng tới khả năng học liên tục từ dữ liệu vận hành, cập nhật bộ nhớ, tự phản tư sau mỗi lần xử lý và sử dụng các cơ chế phản hồi để cải thiện chất lượng chẩn đoán theo thời gian. Mục tiêu cuối cùng là hỗ trợ kỹ sư mạng trong việc phát hiện sự cố, phân tích nguyên nhân gốc rễ và đề xuất hướng xử lý phù hợp, đồng thời đảm bảo kết quả đưa ra có tính giải thích và có thể hỗ trợ quá trình ra quyết định trong thực tế.

1.2 Phát biểu bài toán

Mục tiêu của dự án là xây dựng một hệ thống **Self-Evolving AI Agents** hỗ trợ xử lý sự cố mạng viễn thông, trong đó agent có khả năng tiếp nhận dữ liệu vận hành mạng, phân tích các dấu hiệu bất thường, xác định nguyên nhân có khả năng cao nhất và đề xuất hướng xử lý phù hợp. Khác với các hệ thống chẩn đoán tĩnh, hệ thống được đề xuất cần có khả năng cải thiện theo thời gian thông qua việc học từ lịch sử xử lý, phản hồi của người dùng và kết quả đánh giá sau mỗi lần tương tác.

Cụ thể, bài toán được xem xét trong dự án này bao gồm ba nhóm nhiệm vụ chính. Thứ nhất, hệ thống cần phân tích dữ liệu đầu vào mô tả trạng thái mạng, bao gồm các chỉ số hiệu năng, thông tin cảnh báo, tham số cấu hình và các dữ liệu liên quan đến thiết bị hoặc topology mạng. Thứ hai, hệ thống cần thực hiện suy luận để xác định nguyên nhân gốc rễ của sự cố dựa trên các triệu chứng quan sát được. Thứ ba, hệ thống cần ghi nhận kết quả xử lý, phản hồi và các kinh nghiệm phát sinh để cập nhật bộ nhớ hoặc chiến lược suy luận, từ đó nâng cao hiệu quả xử lý trong các tình huống tương tự về sau.

Đầu vào và đầu ra của bài toán được định nghĩa như sau:

Đầu vào: Dữ liệu vận hành và giám sát mạng viễn thông, có thể bao gồm log hệ thống, cảnh báo mạng, chỉ số hiệu năng, thông tin cấu hình thiết bị, dữ liệu topology, lịch sử xử lý sự cố và phản hồi từ người dùng hoặc chuyên gia.

Đầu ra: Hệ thống AI Agent có khả năng hỗ trợ kỹ sư mạng thực hiện các tác vụ chính: phát hiện hoặc nhận diện sự cố, phân tích nguyên nhân gốc rễ, đưa ra giải thích cho quá trình suy luận, đề xuất hướng khắc phục và cập nhật kinh nghiệm xử lý nhằm cải thiện hiệu quả trong các lần chẩn đoán tiếp theo.

1.3 Đóng góp của dự án

Đóng góp chính của dự án nằm ở việc nghiên cứu và phát triển một hệ thống tự động chẩn đoán sự cố mạng truyền thông dựa trên hướng tiếp cận **Self-Evolving AI Agents** tích hợp trên nền tảng benchmark **NIKA**. Cụ thể hơn, các đóng góp chính của dự án bao gồm:

- ****Xây dựng quy trình chẩn đoán sự cố trên nền tảng NIKA:**** Nghiên cứu và ứng dụng môi trường giả lập mạng Kathará để triển khai các kịch bản sự cố phức tạp (như BGP ASN mismatch, link down, host crash, flow rule loop). Đồng thời thiết lập các AI Agent chẩn đoán hoạt động dựa trên các luồng suy luận phổ biến như ReAct, Plan & Execute, và Reflexion.
- ****Đề xuất và hiện thực hóa Mô-đun Bộ Nhớ Quy Trình Tiên Hóa (Composable Procedural Memory):**** Thiết kế bộ lưu trữ dữ liệu lai giữa PostgreSQL (quan hệ nguyên tử) và Qdrant (vector index). Ứng dụng các cơ chế truy xuất thuộc tính ngữ cảnh (MemInsight), nén vết chẩn đoán (LightMem) và liên kết đồ thị (A-Mem) để giúp agent tích lũy kinh nghiệm chẩn đoán hiệu quả. Đồng thời thiết kế cơ chế score-gated validation dựa trên điểm Evaluator của NIKA nhằm bảo vệ bộ nhớ khỏi hiện tượng học vẹt đáp án benchmark.
- ****Đề xuất và hiện thực hóa Mô-đun Tiên Hóa Công Cụ (Tool Evolution):**** Thiết kế cơ chế cho phép AI Agent tự hoàn thiện tài liệu hướng dẫn công cụ (Mastery) và sinh các công cụ chẩn đoán mới ở test-time (composite workflows và generated Python helpers). Xây dựng cơ chế xác thực an toàn mã nguồn sinh ra bằng cách phân tích AST và thực thi trong môi trường Docker sandbox cô lập.
- ****Đánh giá thực nghiệm chi tiết và sâu sắc:**** Thực hiện kiểm thử agent tự tiến hóa

trên 30 sự cố chọn lọc từ benchmark NIKA, so sánh toàn diện với baseline về các chỉ số: độ chính xác chẩn đoán (detection, localization, RCA), số bước chẩn đoán, số lượng công cụ và lượng token tiêu thụ.

- ****Phân tích thách thức trong thực tế:**** Chỉ ra các thách thức còn tồn tại khi áp dụng Self-Evolving AI Agents vào môi trường mạng viễn thông thực tế, chẳng hạn như chất lượng dữ liệu vận hành, độ tin cậy của phản hồi, rủi ro học sai từ kinh nghiệm quá khứ, khả năng kiểm soát quá trình tự cập nhật và yêu cầu đảm bảo an toàn trong các hệ thống quan trọng.

Chương 2

Kiến thức nền tảng

Chương này trình bày những kiến thức nền tảng cần thiết cho việc nghiên cứu và xây dựng hệ thống Self-Evolving AI Agents trong bài toán xử lý sự cố mạng viễn thông. Các nội dung chính bao gồm: tổng quan về mạng viễn thông và bài toán xử lý sự cố mạng, phân tích nguyên nhân gốc rễ, mô hình ngôn ngữ lớn, AI Agent, Self-Evolving AI Agents, cùng với các cơ chế tự cải thiện như vòng lặp phản hồi, tự phản tư, cập nhật bộ nhớ và học tăng cường.

2.1 Mạng viễn thông

Mạng viễn thông là hệ thống hạ tầng cho phép truyền tải thông tin giữa các thiết bị, người dùng và dịch vụ thông qua nhiều công nghệ truyền dẫn khác nhau. Trong thực tế, mạng viễn thông bao gồm nhiều thành phần như thiết bị truy nhập vô tuyến, trạm gốc, đường truyền, thiết bị lõi mạng, phần mềm điều khiển, giao thức truyền thông và các hệ thống giám sát vận hành. Các thành phần này phối hợp với nhau nhằm đảm bảo quá trình kết nối, truyền dữ liệu và cung cấp dịch vụ diễn ra ổn định.

Trong các mạng di động hiện đại như 4G và 5G, độ phức tạp của hệ thống ngày càng tăng do số lượng thiết bị lớn, cấu hình mạng đa dạng, lưu lượng thay đổi liên tục và yêu cầu chất lượng dịch vụ ngày càng cao. Một thay đổi nhỏ trong cấu hình hoặc một lỗi tại một thành phần có thể gây ảnh hưởng dây chuyền đến nhiều thành phần khác. Vì vậy, việc giám sát, phát hiện và xử lý sự cố mạng viễn thông trở thành một nhiệm vụ quan trọng trong quá trình vận hành mạng.

Bảng 2.1 trình bày một số bài toán tiêu biểu trong xử lý sự cố mạng viễn thông. Các bài toán này thường được thực hiện theo một quy trình liên tục, từ phát hiện sự cố, phân tích nguyên nhân, đánh giá tác động cho đến đề xuất hành động khắc phục.

Bảng 2.1: Các bài toán tiêu biểu trong xử lý sự cố mạng viễn thông

- 1. Phát hiện sự cố:** Mục tiêu là xác định hệ thống mạng có xuất hiện lỗi hoặc dấu hiệu bất thường hay không. Đầu vào thường bao gồm log hệ thống, cảnh báo, chỉ số hiệu năng mạng và dữ liệu giám sát theo thời gian. Các metric thường dùng gồm Accuracy, Precision, Recall và F1-score.
- 2. Phân tích nguyên nhân gốc rễ:** Mục tiêu là xác định nguyên nhân chính gây ra sự cố hoặc triệu chứng bất thường trong mạng. Đầu vào thường bao gồm cảnh báo, log, KPI, dữ liệu cấu hình, topology mạng và lịch sử sự cố. Các metric thường dùng gồm Top-k Accuracy, Precision, Recall và F1-score.
- 3. Đánh giá tác động:** Mục tiêu là xác định phạm vi, mức độ nghiêm trọng và đối tượng bị ảnh hưởng bởi sự cố. Đầu vào có thể bao gồm thông tin dịch vụ, dữ liệu người dùng, topology mạng, vùng phủ sóng và KPI sau sự cố. Các metric thường dùng gồm MAE, RMSE, Accuracy và Severity Score.
- 4. Đề xuất khắc phục:** Mục tiêu là đưa ra hành động xử lý hoặc khuyến nghị vận hành nhằm khôi phục trạng thái mạng. Đầu vào thường bao gồm nguyên nhân gốc rễ, lịch sử xử lý, tài liệu kỹ thuật và trạng thái mạng hiện tại. Các metric thường dùng gồm Success Rate, Recovery Rate và MTTR.

2.2 Phân tích nguyên nhân gốc rễ

Phân tích nguyên nhân gốc rễ, hay Root Cause Analysis (RCA), là nhiệm vụ xác định nguyên nhân chính gây ra một hoặc nhiều triệu chứng bất thường trong hệ thống. Trong bối cảnh mạng viễn thông, RCA không chỉ dừng lại ở việc nhận biết rằng mạng đang có lỗi, mà còn cần trả lời câu hỏi: lỗi bắt nguồn từ đâu, vì sao lỗi xảy ra và cần xử lý như thế nào.

Về mặt tổng quát, bài toán RCA có thể được mô tả như sau. Cho một tập dữ liệu mô tả trạng thái mạng, bao gồm các tham số cấu hình, dữ liệu quan sát và triệu chứng bất thường, mục tiêu là xác định nguyên nhân có khả năng cao nhất gây ra triệu chứng đó. Nếu ký hiệu U là tập tham số cấu hình mạng, Y_t là dữ liệu quan sát tại thời điểm t , s_t là triệu chứng được phát hiện và C là tập các nguyên nhân có thể xảy ra, thì nhiệm vụ RCA là tìm ra nguyên nhân $\hat{c}$ phù hợp nhất để giải thích cho triệu chứng quan sát được.

$$\hat{c} = \operatorname{argmax}_{c \in C} p(c \mid U, Y_t, s_t) \quad (2.1)$$

Trong đó, $p(c \mid U, Y_t, s_t)$ biểu diễn xác suất của nguyên nhân c khi đã biết cấu hình

mạng, dữ liệu quan sát và triệu chứng hiện tại. Cách mô hình hóa này cho thấy RCA là một bài toán suy luận ngược, từ các biểu hiện quan sát được để tìm ra nguyên nhân tiềm ẩn. Đây là bài toán khó do dữ liệu mạng thường có nhiều chiều, chứa nhiễu, thay đổi theo thời gian và có nhiều quan hệ phụ thuộc phức tạp giữa các thành phần mạng.

Trong hệ thống được nghiên cứu, RCA đóng vai trò là một tác vụ trung tâm. Agent cần phân tích dữ liệu đầu vào, suy luận nguyên nhân gốc rễ, đưa ra lời giải thích và ghi nhận kết quả để cải thiện khả năng xử lý trong các tình huống tương tự về sau.

2.3 Mô hình ngôn ngữ lớn

Mô hình ngôn ngữ lớn, hay Large Language Model (LLM), là các mô hình học sâu được huấn luyện trên lượng dữ liệu văn bản rất lớn nhằm học cách hiểu, sinh và suy luận trên ngôn ngữ tự nhiên. Các LLM hiện đại có khả năng thực hiện nhiều tác vụ khác nhau như trả lời câu hỏi, tóm tắt văn bản, sinh mã nguồn, lập kế hoạch, suy luận nhiều bước và sử dụng công cụ thông qua lời gọi API.

Trong bài toán xử lý sự cố mạng viễn thông, LLM có một số ưu điểm quan trọng. Thứ nhất, LLM có thể xử lý dữ liệu dạng văn bản như log hệ thống, cảnh báo lỗi, mô tả sự cố hoặc tài liệu kỹ thuật. Thứ hai, LLM có thể sinh lời giải thích bằng ngôn ngữ tự nhiên, giúp kỹ sư mạng hiểu rõ hơn quá trình chẩn đoán. Thứ ba, LLM có thể kết hợp tri thức miền với dữ liệu quan sát để thực hiện suy luận nhiều bước.

Tuy nhiên, LLM cũng có những hạn chế nhất định. Các mô hình tổng quát có thể thiếu tri thức chuyên sâu về mạng viễn thông, dễ sinh ra câu trả lời thiếu nhất quán hoặc không đủ độ chính xác cho các quyết định vận hành quan trọng. Vì vậy, trong các hệ thống thực tế, LLM thường cần được kết hợp với dữ liệu miền, công cụ kiểm chứng, bộ nhớ, cơ chế phản hồi hoặc các phương pháp huấn luyện bổ sung để nâng cao độ tin cậy.

2.4 AI Agent

AI Agent là một hệ thống trí tuệ nhân tạo có khả năng quan sát môi trường, lập kế hoạch, đưa ra hành động và điều chỉnh hành vi để đạt được mục tiêu nhất định. Khác với một mô hình chỉ sinh phản hồi đơn lẻ, agent thường được thiết kế như một hệ thống gồm nhiều thành phần, bao gồm mô hình nền, bộ nhớ, tập công cụ, bộ lập kế hoạch và cơ chế đánh giá kết quả.

Một AI Agent có thể được mô tả thông qua quá trình tương tác với môi trường. Agent nhận quan sát từ môi trường, sử dụng mô hình suy luận để quyết định hành động,

thực thi hành động thông qua công cụ hoặc API, sau đó nhận phản hồi từ môi trường để tiếp tục vòng lặp xử lý. Trong bài toán xử lý sự cố mạng viễn thông, môi trường có thể là hệ thống vận hành mạng, dữ liệu log, cảnh báo, dashboard giám sát hoặc phản hồi từ kỹ sư vận hành.

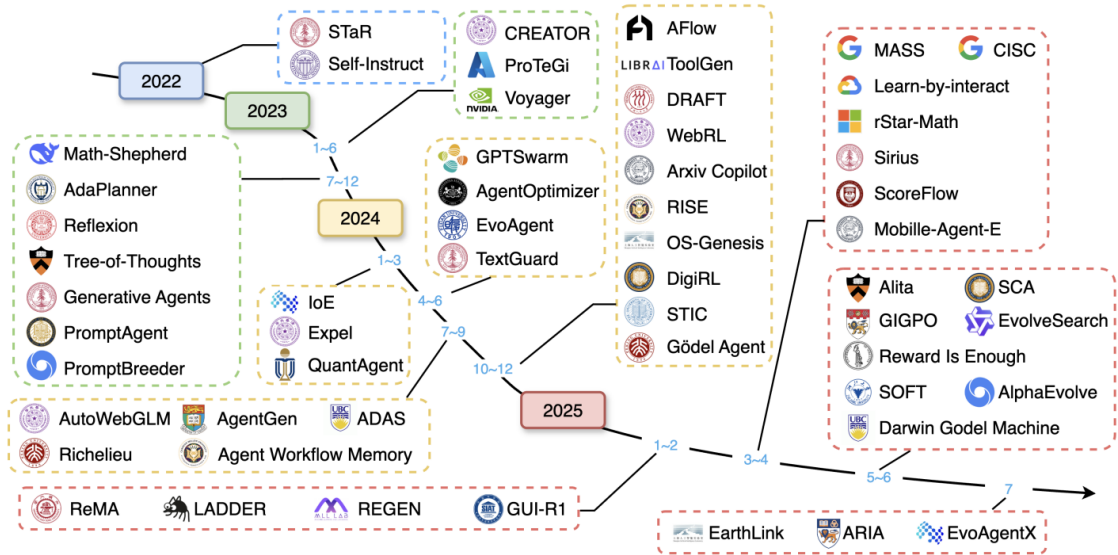
Các hành động của agent trong hệ thống xử lý sự cố có thể bao gồm: truy vấn dữ liệu log, phân tích chỉ số hiệu năng, tìm kiếm tri thức liên quan, xác định nguyên nhân gốc rễ, sinh báo cáo chẩn đoán hoặc đề xuất phương án xử lý. Nhờ khả năng kết hợp suy luận ngôn ngữ với sử dụng công cụ, AI Agent phù hợp với các bài toán phức tạp, cần nhiều bước xử lý và cần tương tác với nhiều nguồn dữ liệu khác nhau.

2.5 Self-Evolving AI Agents

Self-Evolving AI Agents là các hệ thống agent có khả năng tự cải thiện dựa trên dữ liệu, phản hồi và kinh nghiệm thu được trong quá trình hoạt động. Khác với các agent thông thường, vốn chủ yếu vận hành dựa trên mô hình, prompt, bộ nhớ, công cụ hoặc quy trình xử lý được thiết kế cố định từ ban đầu, self-evolving agent hướng tới khả năng tự điều chỉnh một hoặc nhiều thành phần bên trong hệ thống nhằm nâng cao hiệu quả xử lý trong các nhiệm vụ tương lai.

Về bản chất, self-evolving agent không chỉ giải quyết một nhiệm vụ đơn lẻ, mà còn khai thác chính quá trình giải quyết nhiệm vụ đó như một nguồn kinh nghiệm học tập. Các thông tin như dữ liệu đầu vào, chuỗi suy luận, hành động đã thực hiện, công cụ đã sử dụng, kết quả đầu ra, phản hồi từ môi trường hoặc đánh giá từ người dùng đều có thể được sử dụng để cải thiện agent. Nhờ đó, agent có thể rút kinh nghiệm từ các lần xử lý trước, tránh lặp lại lỗi sai và điều chỉnh chiến lược suy luận khi gặp các tình huống tương tự.

Trong bài toán xử lý sự cố mạng viễn thông, khả năng tự cải thiện có ý nghĩa đặc biệt quan trọng. Mạng viễn thông là một môi trường động, trong đó cấu hình thiết bị, lưu lượng người dùng, điều kiện truyền dẫn, trạng thái dịch vụ và các loại sự cố có thể thay đổi liên tục theo thời gian. Một chiến lược chẩn đoán hiệu quả ở thời điểm hiện tại có thể không còn phù hợp khi hệ thống mạng thay đổi hoặc khi xuất hiện các dạng lỗi mới. Vì vậy, self-evolving agent có thể được xem là một hướng tiếp cận phù hợp để xây dựng hệ thống hỗ trợ kỹ sư mạng trong việc phát hiện sự cố, phân tích nguyên nhân gốc rễ và đề xuất phương án xử lý một cách thích nghi hơn.



Hình 2.1: Bức tranh phát triển của một số framework tiêu biểu trong lĩnh vực Self-Evolving AI Agents giai đoạn 2022–2025. Hình được tổ chức theo trình tự thời gian, thể hiện các mốc nghiên cứu quan trọng trong quá trình phát triển agent có khả năng lập kế hoạch tự động, sử dụng công cụ và tự cải thiện liên tục.

2.5.1 Các thành phần có thể tiến hóa

Một câu hỏi quan trọng khi thiết kế self-evolving agent là: thành phần nào của agent có thể được cải thiện theo thời gian. Các thành phần thường được xem xét bao gồm mô hình, ngữ cảnh, bộ nhớ, công cụ và kiến trúc agent.

Mô hình là thành phần lõi quyết định khả năng hiểu, suy luận và sinh phản hồi của agent. Tiến hóa ở mức mô hình thường liên quan đến việc cập nhật tham số, fine-tuning hoặc tối ưu chính sách dựa trên dữ liệu mới và phản hồi thu được. Trong bài toán xử lý sự cố mạng viễn thông, mô hình có thể được cải thiện bằng cách học từ các ca chẩn đoán đúng, các lỗi dự đoán sai hoặc các lời giải thích đã được chuyên gia xác nhận.

Ngữ cảnh bao gồm prompt, hướng dẫn, thông tin truy xuất và các dữ liệu được cung cấp cho agent tại thời điểm suy luận. Tiến hóa ngữ cảnh có thể hiểu là việc cải thiện cách tổ chức đầu vào, cách đặt câu hỏi, cách lựa chọn thông tin liên quan hoặc cách xây dựng prompt để agent đưa ra kết quả tốt hơn. Với bài toán RCA, ngữ cảnh tốt cần giúp agent tập trung vào các dấu hiệu quan trọng như cảnh báo, log, KPI bất thường, cấu hình thiết bị và quan hệ topology.

Bộ nhớ giúp agent lưu trữ và truy xuất kinh nghiệm từ các lần xử lý trước. Bộ nhớ có thể bao gồm lịch sử sự cố, kết quả chẩn đoán, phản hồi từ chuyên gia, quy tắc suy

luận, tài liệu kỹ thuật hoặc các mẫu lỗi thường gặp. Tiến hóa bộ nhớ không chỉ là lưu thêm thông tin mới, mà còn bao gồm cập nhật, hợp nhất hoặc loại bỏ những thông tin sai, trùng lặp hoặc không còn phù hợp.

Công cụ là các chức năng hoặc mô-đun bên ngoài mà agent có thể sử dụng để hoàn thành nhiệm vụ. Trong hệ thống xử lý sự cố mạng viễn thông, công cụ có thể là mô-đun truy vấn log, kiểm tra KPI, phân tích topology, tìm kiếm tài liệu kỹ thuật hoặc đánh giá kết quả RCA. Tiến hóa công cụ có thể bao gồm việc học cách chọn công cụ phù hợp hơn, cải thiện thứ tự sử dụng công cụ hoặc bổ sung công cụ mới khi agent phát hiện thiếu năng lực xử lý.

Kiến trúc agent mô tả cách các thành phần trong hệ thống được tổ chức và phối hợp với nhau. Ở mức kiến trúc, quá trình tiến hóa có thể là thay đổi luồng xử lý, thêm bước kiểm chứng, phân chia vai trò cho nhiều agent con hoặc điều chỉnh cách phối hợp giữa các agent. Ví dụ, một hệ thống có thể gồm agent phân tích dữ liệu, agent suy luận nguyên nhân, agent kiểm chứng kết quả và agent đề xuất phương án khắc phục.

2.5.2 Thời điểm tiến hóa

Bên cạnh câu hỏi thành phần nào cần tiến hóa, một vấn đề quan trọng khác là quá trình tiến hóa diễn ra vào thời điểm nào. Có thể chia thời điểm tiến hóa thành hai nhóm chính: tiến hóa trong khi xử lý nhiệm vụ và tiến hóa sau khi nhiệm vụ kết thúc.

Intra-test-time evolution là quá trình agent tự điều chỉnh trong khi đang giải quyết một nhiệm vụ cụ thể. Khi phát hiện kết quả chưa chắc chắn, thiếu dữ liệu hoặc tồn tại nhiều giả thuyết cạnh tranh, agent có thể truy xuất thêm thông tin, thay đổi kế hoạch, kiểm tra lại suy luận hoặc cập nhật tạm thời ngữ cảnh để cải thiện kết quả hiện tại. Trong xử lý sự cố mạng viễn thông, điều này tương ứng với việc agent đang phân tích một sự cố nhưng tự nhận thấy còn thiếu log, thiếu thông tin cấu hình hoặc chưa đủ bằng chứng để kết luận nguyên nhân gốc rễ.

Inter-test-time evolution là quá trình agent học sau khi một nhiệm vụ đã hoàn thành. Agent sử dụng lịch sử xử lý, kết quả đúng sai và phản hồi từ người dùng để cải thiện cho các nhiệm vụ tiếp theo. Đây là cơ chế phù hợp với môi trường vận hành mạng, nơi mỗi ca sự cố sau khi được xử lý có thể trở thành một kinh nghiệm mới giúp hệ thống chẩn đoán tốt hơn trong tương lai.

2.5.3 Các phương thức tiến hóa thường gặp

Dựa trên cách agent tiếp nhận tín hiệu học tập và cập nhật hành vi, các phương thức tiến hóa thường gặp có thể được chia thành một số nhóm chính, bao gồm tiến hóa dựa trên phần thưởng, tiến hóa dựa trên tự phản tư, tiến hóa bộ nhớ, học từ minh họa và tiến hóa dựa trên quần thể.

2.5.3.1 Tiến hóa dựa trên phần thưởng

Tiến hóa dựa trên phần thưởng là phương thức trong đó agent cải thiện hành vi thông qua các tín hiệu đánh giá. Tín hiệu này có thể cho biết kết quả của agent là tốt hay chưa tốt, nguyên nhân dự đoán có đúng hay không, lời giải thích có hợp lý hay không hoặc hành động khắc phục có mang lại hiệu quả hay không.

Trong nhóm này, vòng lặp phản hồi là cơ chế nền tảng. Agent thực hiện một nhiệm vụ, sinh kết quả, nhận phản hồi và sử dụng phản hồi đó để điều chỉnh hành vi trong các lần xử lý sau. Một vòng lặp phản hồi cơ bản có thể được mô tả như sau:

Input – Agent Reasoning – Output – Feedback – Update

Trong đó, bước *Update* có thể là cập nhật bộ nhớ, điều chỉnh prompt, thay đổi chiến lược suy luận hoặc huấn luyện lại một phần mô hình. Với bài toán RCA, phản hồi có thể đến từ chuyên gia mạng, từ bộ đánh giá tự động hoặc từ kết quả thực tế sau khi áp dụng phương án khắc phục.

Một dạng quan trọng của tiến hóa dựa trên phần thưởng là học tăng cường. Trong học tăng cường, agent học cách lựa chọn hành động thông qua việc tương tác với môi trường và nhận phần thưởng tương ứng. Nếu agent xác định đúng nguyên nhân gốc rễ hoặc đưa ra khuyến nghị xử lý hiệu quả, hệ thống có thể gán phần thưởng cao. Ngược lại, nếu agent bỏ sót nguyên nhân, chọn sai lỗi hoặc đưa ra giải thích không phù hợp, phần thưởng sẽ thấp hơn. Theo thời gian, agent học cách ưu tiên các chiến lược có khả năng tạo ra kết quả tốt hơn.

Ngoài phần thưởng dạng số, phản hồi cũng có thể tồn tại dưới dạng ngôn ngữ tự nhiên. Ví dụ, chuyên gia có thể nhận xét rằng agent đã đúng khi kiểm tra KPI nhưng bỏ sót quan hệ topology giữa hai thiết bị. Loại phản hồi này giúp agent hiểu rõ hơn lỗi trong quá trình suy luận, từ đó cải thiện prompt, bộ nhớ hoặc chiến lược phân tích ở các lần sau.

2.5.3.2 Tiến hóa dựa trên tự phản tư

Tự phản tư là cơ chế cho phép agent tự đánh giá lại quá trình suy luận và kết quả của chính mình. Sau khi hoàn thành một tác vụ, agent có thể phân tích xem kết luận đã đủ bằng chứng chưa, có bỏ sót dữ liệu nào không, giả thuyết nào đã bị loại bỏ sai hoặc lời giải thích có phù hợp với dữ liệu quan sát hay không.

Trong bài toán RCA, tự phản tư giúp agent phát hiện các lỗi như chọn nguyên nhân chưa đủ bằng chứng, suy luận nhầm quan hệ nhân quả, bỏ sót cảnh báo quan trọng hoặc đưa ra lời giải thích quá chung chung. Kết quả phản tư có thể được lưu vào bộ nhớ dưới dạng kinh nghiệm hoặc quy tắc suy luận để tránh lặp lại sai lầm.

Ví dụ, nếu agent dự đoán nguyên nhân là lỗi handover nhưng phản hồi từ chuyên gia cho biết nguyên nhân thực tế là nhiễu do cell lân cận, agent có thể ghi nhận rằng trong các trường hợp suy giảm thông lượng đi kèm chỉ số nhiễu cao và có cell lân cận đồng tần số, cần ưu tiên kiểm tra giả thuyết về nhiễu trước khi kết luận lỗi handover. Như vậy, tự phản tư giúp biến lỗi sai thành dữ liệu học tập cho tương lai.

2.5.3.3 Tiến hóa bộ nhớ

Tiến hóa bộ nhớ là phương thức trong đó agent cải thiện thông qua việc lưu trữ, cập nhật và truy xuất kinh nghiệm từ các lần tương tác trước. Bộ nhớ giúp agent không phải xử lý mọi tình huống từ đầu, mà có thể tận dụng các trường hợp tương tự đã từng gặp.

Trong hệ thống xử lý sự cố mạng viễn thông, bộ nhớ có thể lưu các ca sự cố đã xử lý, nguyên nhân được xác nhận, dấu hiệu bất thường đi kèm, phương án khắc phục, phản hồi từ chuyên gia và kết quả sau khắc phục. Khi gặp một sự cố mới, agent có thể truy xuất các ca tương tự để hỗ trợ suy luận.

Một cơ chế cập nhật bộ nhớ thường bao gồm ba thao tác chính. Thứ nhất là **thêm mới**, tức là lưu lại một kinh nghiệm, sự cố hoặc phản hồi mới có giá trị cho tương lai. Thứ hai là **cập nhật**, tức là điều chỉnh một ghi nhớ cũ khi có thông tin chính xác hơn hoặc khi điều kiện vận hành thay đổi. Thứ ba là **loại bỏ**, tức là xóa hoặc giảm ưu tiên các thông tin sai, lỗi thời hoặc không còn phù hợp.

Tuy nhiên, tiến hóa bộ nhớ cần được kiểm soát cẩn thận. Nếu agent lưu lại phản hồi sai hoặc kết luận chưa được kiểm chứng, bộ nhớ có thể làm giảm chất lượng suy luận trong tương lai. Vì vậy, với các hệ thống mạng quan trọng, thông tin đưa vào bộ nhớ nên được đánh giá hoặc xác nhận trước khi trở thành tri thức lâu dài.

2.5.3.4 Học từ minh họa và kinh nghiệm mẫu

Học từ minh họa, hay imitation and demonstration learning, là phương thức trong đó agent cải thiện bằng cách học từ các ví dụ xử lý tốt. Thay vì chỉ nhận phần thưởng đúng sai, agent được cung cấp một chuỗi xử lý hoàn chỉnh, bao gồm cách phân tích dữ liệu, cách loại trừ giả thuyết sai, cách lựa chọn nguyên nhân và cách trình bày lời giải thích.

Trong self-evolving agents, các minh họa này không nhất thiết phải đến từ con người. Chúng có thể được tạo ra từ chính agent sau khi giải đúng một bài toán, từ một agent mạnh hơn, từ nhiều agent cùng thảo luận hoặc từ dữ liệu đã được chuyên gia xác nhận. Agent sau đó học lại từ các trajectory chất lượng cao để cải thiện năng lực xử lý.

Trong bài toán xử lý sự cố mạng viễn thông, phương thức này có thể được áp dụng bằng cách thu thập các ca RCA mẫu đã được kỹ sư mạng xác nhận. Mỗi ca mẫu có thể bao gồm dữ liệu đầu vào, các bước phân tích, nguyên nhân gốc rễ, bằng chứng hỗ trợ và phương án xử lý. Khi học từ các ví dụ này, agent không chỉ học đáp án cuối cùng mà còn học cách suy luận theo từng bước, từ đó cải thiện khả năng giải thích và độ tin cậy của kết quả.

2.5.3.5 Tiến hóa dựa trên quần thể

Tiến hóa dựa trên quần thể là phương thức trong đó hệ thống duy trì nhiều biến thể agent, nhiều chiến lược xử lý hoặc nhiều cấu hình kiến trúc khác nhau. Các biến thể này được đánh giá trên cùng một nhóm nhiệm vụ, sau đó những biến thể hiệu quả hơn được giữ lại, kết hợp hoặc tiếp tục cải tiến. Cách tiếp cận này lấy cảm hứng từ quá trình chọn lọc và tiến hóa tự nhiên.

Ở mức đơn agent, hệ thống có thể thử nhiều phiên bản prompt, nhiều chiến lược suy luận hoặc nhiều cách sử dụng công cụ khác nhau. Phiên bản nào tạo ra kết quả tốt hơn sẽ được ưu tiên sử dụng trong các lần sau. Ở mức đa agent, hệ thống có thể thử nhiều cách phân chia vai trò, chẳng hạn agent phân tích log, agent kiểm tra KPI, agent suy luận nguyên nhân và agent phản biện kết quả. Cách phối hợp nào đạt hiệu quả cao hơn sẽ được giữ lại.

Trong bối cảnh xử lý sự cố mạng viễn thông, tiến hóa dựa trên quần thể có thể giúp tìm ra quy trình chẩn đoán phù hợp hơn cho từng loại sự cố. Ví dụ, với sự cố suy giảm thông lượng, một workflow ưu tiên phân tích KPI vô tuyến có thể hiệu quả hơn. Trong khi đó, với sự cố mất dịch vụ hàng loạt, workflow ưu tiên phân tích topology và alarm correlation có thể phù hợp hơn. Việc duy trì nhiều biến thể cho phép hệ thống linh hoạt

hơn thay vì phụ thuộc vào một quy trình cố định.

2.5.4 Các công trình nghiên cứu tự tiến hóa nền tảng

Trong nghiên cứu này, các mô-đun cốt lõi được thiết kế dựa trên sự kế thừa và kết hợp các ưu điểm từ một số nghiên cứu tự tiến hóa nổi bật trong lĩnh vực AI Agent, được chia thành hai nhóm chính: tiến hóa bộ nhớ và tiến hóa công cụ.

2.5.4.1 Các nghiên cứu về Tiến hóa Bộ nhớ (Memory Evolution)

- **MemInsight:** Đề xuất cơ chế **Attribute-aware retrieval** (truy xuất nhận biết thuộc tính). MemInsight nhận định rằng việc truy xuất bộ nhớ chỉ dựa trên khoảng cách cosine của vector nhúng ngữ nghĩa (semantic similarity) dễ dẫn đến hiện tượng nhiễu thông tin, đặc biệt trong các miền chuyên sâu. Bằng cách gán thêm các thuộc tính ngữ cảnh cụ thể (contextual attributes) cho các mẫu bộ nhớ (như các thiết bị bị ảnh hưởng, loại giao thức, dạng triệu chứng), hệ thống có thể kết hợp lọc thuộc tính trước khi tính toán độ tương đồng, giúp chọn lọc chính xác các bài học phù hợp nhất với ngữ cảnh sự cố hiện tại.
- **LightMem:** Tập trung vào việc nén vết chẩn đoán và cập nhật ngoại tuyến (**offline consolidation / trajectory compaction**). Vết tương tác (trajectory) của AI Agent với môi trường thường rất dài, chứa nhiều bước gọi công cụ trung gian hoặc thông tin thừa. LightMem đề xuất phân đoạn vết chẩn đoán này theo các chủ đề chuyên biệt (diagnostic topics) và tóm tắt chúng thành các bài học ngắn gọn. Đồng thời, việc cập nhật bộ nhớ được tách biệt khỏi quá trình thực thi thời gian thực (được cập nhật ở pha "sleep-time" hoặc sau khi hoàn thành công việc), đảm bảo hiệu năng và giảm tải tài nguyên trong lúc agent hoạt động.
- **A-Mem:** Xây dựng đồ thị bộ nhớ nguyên tử (**atomic memory graph**). Thay vì lưu trữ các đoạn hội thoại nguyên bản, A-Mem đề xuất chuyển hóa kinh nghiệm thành các ghi chú nguyên tử ngắn gọn (atomic notes), độc lập và có tính tái sử dụng cao. Các ghi chú này được liên kết với nhau bằng các quan hệ lô-gích như **supports** (hỗ trợ), **refines** (làm mịn/chi tiết hóa), và **contradicts** (mâu thuẫn). Sự tiến hóa của bộ nhớ diễn ra thông qua việc điều chỉnh các trọng số tin cậy (confidence) của các nút và mối liên kết trên đồ thị, đồng thời loại bỏ hoặc thay thế các tri thức lỗi thời bằng các tri thức mới có độ tin cậy cao hơn.

2.5.4.2 Các nghiên cứu về Tiến hóa Công cụ (Tool Evolution)

- ****DRAFT:**** Tập trung vào việc học cách sử dụng các công cụ có sẵn tốt hơn thông qua tinh chỉnh tài liệu mô tả công cụ (*documentation refinement / tool mastery*). DRAFT không thay đổi mã nguồn thực thi của công cụ nguyên thủy (primitive tools) mà sử dụng một vòng lặp: Explorer thực thi thử nghiệm công cụ, Analyzer phân tích kết quả phản hồi lỗi, và Rewriter viết lại tài liệu hướng dẫn (mô tả tham số, điều kiện tiên đề, giải nghĩa kết quả trả về). Điều này giúp LLM hiểu sâu hơn các ràng buộc và sử dụng công cụ hiệu quả hơn ở các lần sau.
- ****TTE (Test-Time Tool Evolution):**** Cho phép AI Agent tự sinh các công cụ thực thi mã nguồn Python mới ngay trong thời gian chạy (*test-time tool evolution*). Khi đối mặt với một capability gap (thiếu hụt năng lực chẩn đoán mà các công cụ hiện tại không hỗ trợ), agent có thể tự sinh mã nguồn Python để giải quyết bài toán tính toán cụ thể. Công cụ mới sinh ra phải được kiểm định cú pháp AST và chạy thử nghiệm an toàn trong môi trường ảo hóa Docker sandbox trước khi được thêm vào thư viện công cụ chính thức.
- ****Alita / Alita-G:**** Đề xuất cơ chế tự động tạo và trừu tượng hóa các công cụ Model Context Protocol (MCP) thành các hộp năng lực (capability boxes/MCP Box). Alita nhấn mạnh việc lọc bỏ các bước gọi công cụ lỗi hoặc lặp lại trong các vết tương tác thành công, tổng hợp các tham số biến thiên thành các tham số cấu hình chung để tạo ra các công cụ phức hợp (composite tools) có thể tái sử dụng, giúp đơn giản hóa việc gọi công cụ của agent.

2.5.5 Vai trò trong xử lý sự cố mạng viễn thông

Đối với xử lý sự cố mạng viễn thông, Self-Evolving AI Agents có thể đóng vai trò như một trợ lý thông minh hỗ trợ kỹ sư mạng trong toàn bộ quá trình chẩn đoán. Agent có thể phân tích dữ liệu vận hành, xác định các dấu hiệu bất thường, đề xuất nguyên nhân gốc rễ, giải thích cơ sở suy luận và gợi ý phương án khắc phục.

Điểm khác biệt quan trọng của self-evolving agent nằm ở khả năng cải thiện sau mỗi lần xử lý. Mỗi ca sự cố không chỉ là một nhiệm vụ riêng lẻ mà còn là một nguồn dữ liệu mới cho hệ thống. Khi được thiết kế đúng, agent có thể học từ các ca thành công, rút kinh nghiệm từ các ca thất bại, cập nhật bộ nhớ từ phản hồi của chuyên gia và điều chỉnh chiến lược suy luận khi môi trường mạng thay đổi.

Nhờ đó, Self-Evolving AI Agents mở ra một hướng tiếp cận phù hợp cho các hệ

thông mạng hiện đại, nơi dữ liệu vận hành phức tạp, sự cố có thể phát sinh liên tục và yêu cầu xử lý ngày càng cao. Thay vì chỉ xây dựng một mô hình chẩn đoán tĩnh, hướng tiếp cận này tập trung vào một hệ thống có khả năng thích nghi, học hỏi và nâng cao hiệu quả theo thời gian.

Chương 3

Kiến trúc hệ thống

Chương này trình bày kiến trúc chi tiết của hệ thống tự chẩn đoán sự cố mạng truyền thông dựa trên hướng tiếp cận Self-Evolving AI Agents tích hợp trên nền tảng NIKA. Kiến trúc hệ thống được thiết kế hướng tới khả năng mô phỏng môi trường mạng thực tế, hỗ trợ nhiều luồng suy luận của agent, và đặc biệt là hai mô-đun tự tiến hóa: Mô-đun bộ nhớ quy trình (Memory Module) và Mô-đun tiến hóa công cụ (Tool Evolution Module).

3.1 Kiến trúc tổng quan

Kiến trúc tổng quan của hệ thống bao gồm 4 thành phần chính được kết nối chặt chẽ với nhau như được mô tả trong Hình dưới đây (triển khai dạng luồng tương tác):

1. ****Môi trường mô phỏng mạng (Network Environment):**** Chạy trên nền tảng Kathará ảo hóa dưới dạng các Docker container đại diện cho thiết bị mạng (routers, switches, hosts) kết nối theo các topology phức tạp.
2. ****AI Agent và Workflow chẩn đoán:**** Đóng vai trò là thực thể thực hiện chẩn đoán sự cố, hỗ trợ các workflow suy luận phổ biến như ReAct, Plan & Execute, và Reflexion.
3. ****Hệ thống MCP Servers:**** Cung cấp cầu nối tiêu chuẩn hóa để AI Agent có thể truy vấn thông tin, thực thi lệnh chẩn đoán trên thiết bị mạng ảo hóa, và gửi kết quả chẩn đoán.
4. ****Mô-đun Tự Tiến Hóa (Self-Evolution Modules):**** Gồm mô-đun bộ nhớ quy trình (lưu trữ kinh nghiệm chẩn đoán lâu dài) và mô-đun tiến hóa công cụ (tự sinh công cụ chẩn đoán mới).

3.2 Môi trường mạng ảo hóa và các dịch vụ MCP

Để AI Agent có khả năng tương tác linh hoạt với môi trường mạng giả lập mà không phụ thuộc vào một cấu trúc phần cứng cụ thể, hệ thống ứng dụng giao thức Model Context Protocol (MCP) do Anthropic đề xuất. Nền tảng NIKA thiết kế và khởi chạy 6 máy chủ MCP chuyên biệt trong thư mục `src/nika/service/mcp_server`:

- ****Kathará Base MCP Server:**** Cung cấp các công cụ chẩn đoán cơ bản như kiểm tra kết nối giữa các host (`get_reachability`), ping một cặp thiết bị (`ping_pair`), đo băng thông (`iperf_test`), kiểm tra dịch vụ (`systemctl_ops`), xem cấu hình giao tiếp mạng (`get_host_net_config`), thống kê giao thức (`netstat`) và thực thi shell command trong container (`exec_shell`).
- ****FRR MCP Server:**** Tương tác với phần mềm định tuyến FRRouting chạy trên các router để trích xuất cấu hình định tuyến động OSPF (`frr_get_ospf_conf`), BGP (`frr_get_bgp_conf`) và bảng định tuyến hiện tại (`frr_show_ip_route`).
- ****BMv2 MCP Server:**** Tương tác với các P4 software switches chạy phần mềm BMv2 để đọc dữ liệu telemetry, đọc giá trị counter (`bm2_counter_read`), dump các bảng luồng hành vi (`bm2_table_dump`) và đọc thanh ghi (`bm2_register_r`).
- ****Telemetry MCP Server:**** Truy vấn cơ sở dữ liệu InfluxDB thu thập từ In-band Network Telemetry (INT) để lấy các chỉ số đo lường hiệu năng mạng theo thời gian thực.
- ****Task Management MCP Server:**** Cung cấp công cụ `submit` để agent gửi kết quả chẩn đoán cuối cùng (gồm trạng thái lỗi, thiết bị bị lỗi và phân tích nguyên nhân gốc rễ RCA).
- ****Evolved Diagnostic Toolbox MCP Server:**** Cung cấp công cụ khám phá (`list_evolved`) và thực thi (`execute_evolved_tool`) các công cụ chẩn đoán mới được tiến hóa thành công trong thư viện.

3.3 Các luồng suy luận chẩn đoán của AI Agent

AI Agent trong hệ thống đóng vai trò nhận mô tả tác vụ từ môi trường và sử dụng các công cụ chẩn đoán MCP để tìm ra thiết bị lỗi và nguyên nhân gốc rễ. Hệ thống hỗ trợ ba cơ chế workflow chẩn đoán chính:

- ****ReAct (Reasoning and Acting):**** Agent luân phiên thực hiện bước suy luận ngữ cảnh (Thought) và gọi công cụ chẩn đoán tương ứng (Action) cho đến khi thu thập đủ bằng chứng chẩn đoán để đưa ra quyết định cuối cùng.
- ****Plan & Execute:**** Một bộ lập kế hoạch (Planner) sẽ phân tích bài toán và sinh ra một chuỗi các bước chẩn đoán cần thiết. Một bộ thực thi (Executor) sẽ chạy tuần tự các bước đó. Nếu có lỗi phát sinh hoặc kết quả bất thường, bộ tái lập kế hoạch (Replanner) sẽ điều chỉnh danh sách công việc.
- ****Reflexion:**** Tích hợp vòng lặp phản tư. Sau mỗi lần chẩn đoán thất bại, agent tự phân tích vết suy luận lỗi, rút kinh nghiệm dưới dạng episodic memory và thực hiện chẩn đoán lại với định hướng chiến lược mới nhằm tránh lặp lại sai lầm.

3.4 Mô-đun Bộ Nhớ Quy Trình Tiến Hóa (Memory Module)

Mô-đun Bộ nhớ quy trình giúp lưu trữ tri thức lâu dài cho agent qua các incident chẩn đoán liên tiếp. Mô-đun này được kế thừa và kết hợp các ưu điểm từ 3 công trình lớn: ****MemInsight****, ****LightMem****, và ****A-Mem****.

3.4.1 Cơ chế lưu trữ hỗn hợp

Bộ nhớ quy trình được lưu trữ dưới hai dạng:

1. ****PostgreSQL (Canonical Store):**** Lưu trữ chính thức các bản ghi bộ nhớ, thông tin nguồn gốc (provenance), mức độ tin cậy (confidence), phiên bản, và các liên kết đồ thị giữa các ghi chú.
2. ****Qdrant (Semantic Index):**** Lưu trữ vector nhúng của nội dung ghi nhớ để hỗ trợ tìm kiếm tương đồng ngữ nghĩa. Khi cơ sở dữ liệu Qdrant chưa sẵn sàng, hệ thống tự động fallback sang tìm kiếm văn bản (lexical/full-text search) trên PostgreSQL.

3.4.2 Truy xuất nhận biết thuộc tính (MemInsight-style Retrieval)

Trước mỗi tập chẩn đoán (episode), từ ngữ cảnh công khai ban đầu (public task context), hệ thống sinh ra một tập thuộc tính truy vấn (`query_attributes`) gồm: tên kịch bản mạng, lớp topology, giao thức sử dụng (OSPF, BGP, DHCP, etc.), dịch vụ kích hoạt. Bộ nhớ được truy xuất theo công thức tính điểm kết hợp:

$$\text{Score} = \text{Similarity}_{\text{semantic}} + \text{Bonus}_{\text{attribute}} + \text{Weight}_{\text{confidence}} - \text{Penalty}_{\text{redundancy}} \quad (3.1)$$

Điều này giúp agent lọc bỏ các kinh nghiệm chẩn đoán không liên quan đến cấu trúc mạng hiện tại (ví dụ: không lấy kinh nghiệm BGP Clos khi đang chẩn đoán OSPF Enterprise).

3.4.3 Nén vết tương tác (LightMem-style Compaction)

Sau khi tập chẩn đoán kết thúc, thay vì lưu trữ toàn bộ lịch sử hội thoại dài và nhiều nhiễu (`messages.jsonl`), mô-đun thực hiện quy trình nén vết chẩn đoán:

- Loại bỏ các thông tin nhạy cảm hoặc định danh cụ thể (redact IPv4/IPv6 thành `<ip>`, MAC thành `<mac>`, tên thiết bị thành `<device_A>`).
- Phân đoạn lịch sử theo từng pha chẩn đoán (ví dụ: kiểm tra ban đầu, kiểm tra định tuyến, xác định lỗi).
- LLM đóng vai trò nén thông tin, trích xuất thành các ghi chú nguyên tử ngắn gọn (atomic notes).

3.4.4 Đồ thị liên kết và Cơ chế lọc điểm số (A-Mem & Score-gated Validation)

Các ghi chú nguyên tử mới sinh ra được so sánh với bộ nhớ hiện tại để tạo các liên kết đồ thị:

- ****supports:**** củng cố ghi chú cũ.
- ****refines:**** Chi tiết hóa hoặc bổ sung điều kiện ràng buộc cho ghi chú cũ.
- ****contradicts:**** Mâu thuẫn với ghi chú cũ (xảy ra khi môi trường hoặc cấu hình mạng thay đổi).

Để đảm bảo an toàn và tính khách quan cho benchmark (tránh rò rỉ đáp án), việc cập nhật bộ nhớ phải tuân theo cơ chế ****Score-gated Validation****:

- Chỉ khi NIKA Evaluator chấm điểm chẩn đoán thành công tuyệt đối (độ chính xác $RCA = 1.0$), các ghi chú quy trình chẩn đoán (procedural notes) mới được gán nhãn `validated` và tăng độ tin cậy.
- Nếu chẩn đoán thất bại hoặc thành công một phần, hệ thống chỉ chấp nhận lưu trữ các ghi chú dạng cảnh báo (avoid rules hoặc caution notes, ví dụ: "Không được kết luận lỗi DNS chỉ dựa vào kiểm tra curl"), đồng thời giảm độ tin cậy của các ghi chú đã hướng dẫn sai.

3.5 Mô-đun Tiến Hóa Công Cụ (Tool Evolution Module)

Mô-đun Tiến hóa công cụ cho phép AI Agent tự mở rộng thư viện công cụ của mình để vượt qua các giới hạn năng lực tại test-time. Thư viện công cụ lưu trữ dưới dạng `state.json` gồm 3 thành phần chính:

3.5.1 Mastery mô tả công cụ (DRAFT-like Refinement)

Đối với các công cụ nguyên thủy (primitive tools), hệ thống không thay đổi mã nguồn thực thi mà cập nhật một lớp phủ tài liệu hướng dẫn (ToolMastery) dựa trên kết quả chạy thực tế:

- **preconditions:** Điều kiện phù hợp nhất để gọi công cụ.
- **parameter_guidance:** Hướng dẫn truyền tham số đúng kiểu và đúng ngữ cảnh thiết bị.
- **failure_semantics:** Ý nghĩa của các mã lỗi trả về.

3.5.2 Tổng hợp quy trình phức hợp và sinh mã nguồn Python (TTE & Alita-style)

Khi phát hiện một thiếu hụt năng lực chẩn đoán (capability gap), agent có thể đề xuất 2 dạng công cụ mới:

1. **Composite Workflow:** Chuỗi các bước gọi tuần tự các công cụ nguyên thủy an toàn, được tham số hóa để tái sử dụng (ví dụ: kiểm tra định tuyến tuần tự từ Host -> Router -> Gateway).
2. **Generated Python Helper:** Hàm Python thuần thực hiện các tính toán phức tạp trên dữ liệu chẩn đoán thu thập được (ví dụ: tính toán tỷ lệ mất gói từ các số liệu đếm counter).

3.5.3 Quy trình xác thực an toàn (AST & Sandbox Validation)

Để tránh rủi ro bảo mật hệ thống và đảm bảo công cụ sinh ra hoạt động chính xác, toàn bộ các công cụ Python mới sinh ra đều phải đi qua pipeline xác thực nghiêm ngặt:

1. **Xác thực cấu trúc (AST Verification):** Sử dụng thư viện `ast` của Python để kiểm tra cấu trúc cú pháp, đảm bảo không sử dụng các hàm cấm (như `eval`, `exec`, `open`, `__import__`) và chỉ import các thư viện toán học trong allowlist.

2. ****Thực thi Sandbox (Docker Sandbox execution):**** Chạy thử nghiệm công cụ trong một container Docker cô lập hoàn toàn mạng (`-network none`) với các tham số mẫu. Hàm sinh ra phải chạy thành công và trả về định dạng JSON hợp lệ.
3. ****Xác thực kết quả thực nghiệm:**** Công cụ mới sinh ra ban đầu ở trạng thái thử nghiệm (`candidate`). Nó chỉ được thăng cấp lên chính thức (`promoted`) sau khi hỗ trợ agent chẩn đoán thành công sự cố ở ít nhất 2 ngữ cảnh kịch bản mạng khác nhau.

Chương 4

Thực nghiệm

Chương này trình bày chi tiết về thiết lập thực nghiệm, bộ kịch bản lỗi được sử dụng, và kết quả đánh giá thực tế của hệ thống Self-Evolving AI Agents trên nền tảng NIKA. Các kết quả này chứng minh tính hiệu quả của mô-đun tiến hóa công cụ và bộ nhớ quy trình trong việc nâng cao độ chính xác và hiệu quả chẩn đoán lỗi mạng.

4.1 Thiết lập thực nghiệm

Thực nghiệm được thực hiện trên hệ thống máy cá nhân chạy Linux Ubuntu với cấu hình phần cứng và phần mềm cụ thể như sau:

- **Phần cứng:** CPU Intel Core i7 đa nhân, RAM 16GB, và GPU NVIDIA GeForce RTX 3050 8GB để hỗ trợ thực thi các mô hình cục bộ hoặc xử lý song song.
- **Môi trường ảo hóa:** Docker Engine được cài đặt để chạy các container Kathará mô phỏng mạng, PostgreSQL 15 để lưu trữ bộ nhớ quan hệ, và Qdrant để lưu trữ vector ngữ nghĩa.
- **Mô hình ngôn ngữ lớn:** Sử dụng mô hình `openai/gpt-oss-120b` thông qua NetMind API để đảm bảo khả năng suy luận chẩn đoán mạnh mẽ.
- **Công cụ lập trình:** Dự án được lập trình hoàn toàn bằng ngôn ngữ Python 3.12, sử dụng `uv` để quản lý thư viện và chạy các framework chính như LangGraph và LangChain.

4.2 Bộ kịch bản sự cố và cơ chế chia dữ liệu

Thực nghiệm sử dụng tập con chọn lọc gồm 30 incident khác nhau trích xuất từ kịch bản mạng của NIKA (định nghĩa trong `benchmark/tool_evolution_30.csv`). Các kịch bản này được chia thành 5 nhóm chính (Evolution Streams) để đánh giá khả năng chẩn đoán trên nhiều khía cạnh của mạng truyền thông:

1. ****dns_dhcp_service:**** Gồm các sự cố về dịch vụ cơ bản như lỗi bản ghi DNS (`dns_record_error`), dịch vụ DNS bị dừng (`dns_service_down`), dịch vụ DHCP bị dừng (`dhcp_service_down`), và thiếu cấu hình mạng con trong DHCP (`dhcp_missing_subnet`).
2. ****host_ip_config:**** Gồm các lỗi cấu hình IP trên host như sai IP (`host_incorrect_ip`), mất IP (`host_missing_ip`), sai mặt nạ mạng (`host_incorrect_netmask`), sai gateway mặc định (`host_incorrect_gateway`), trùng IP (`host_ip_conflict`), và xung đột địa chỉ MAC (`mac_address_conflict`).
3. ****link_path:**** Gồm các lỗi liên quan đến đường truyền vật lý như mất kết nối (`link_down`), cáp bị tháo (`link_detach`), link nhấp nháy (`link_flap`), lỗi phân mảnh MTU (`link_fragmentation_disabled`), giới hạn băng thông (`link_bandwidth_throttling`), và tỷ lệ mất gói cao (`link_high_packet_corruption`).
4. ****routing_control:**** Gồm các sự cố cấu hình định tuyến tĩnh hoặc động như sai số hiệu tự trị BGP (`bgp_asn_misconfig`), thiếu quảng bá định tuyến BGP (`bgp_missing_route_advertisement`), dịch vụ FRRouting bị dừng (`frr_service_down`), tấn công giả mạo BGP (`bgp_hijacking`), mất neighbor OSPF (`ospf_neighbor_mismatch`) và sai cấu hình phân vùng OSPF (`ospf_area_misconfiguration`).
5. ****sdn_p4_control:**** Gồm các lỗi trên thiết bị mạng lập trình được và SDN như vòng lặp luật dòng chảy (`flow_rule_loop`), xung đột luật (`flow_rule_shadowing`), lỗi SDN controller bị sập (`sdn_controller_crash`), sai cổng kết nối southbound (`southbound_port_mismatch`), thiếu bảng luồng P4 (`p4_table_entry_missing`) và cấu hình bảng luồng P4 bị sai (`p4_table_entry_misconfig`).

Toàn bộ 30 incident được chia đều thành 3 phân đoạn đánh giá (Splits):

- ****evolution:**** Phân đoạn cho phép agent vừa chẩn đoán vừa cập nhật tự tiến hóa thư viện công cụ và bộ nhớ quy trình (cho phép quyền ghi và tiến hóa).

- ****development:**** Phân đoạn chỉ cho phép đọc thông tin từ thư viện công cụ và bộ nhớ đã tiến hóa trước đó (đánh giá khả năng khai thác kiến thức cũ).
- ****transfer:**** Phân đoạn kiểm tra khả năng chuyển đổi tri thức (transfer learning) khi áp dụng thư viện công cụ đã học sang các kịch bản mạng và lỗi hoàn toàn mới.

4.3 Kết quả thực nghiệm

Bảng 4.1 trình bày kết quả so sánh tổng quan giữa Baseline Agent (không sử dụng cơ chế tiến hóa) và Evolved Agent (kết hợp cả Tool Evolution và Memory Evolution chế độ dual) chạy trên mô hình gpt-oss-120b.

Bảng 4.1: So sánh hiệu năng tổng quan giữa Baseline Agent và Evolved Agent

Chỉ số đánh giá	Baseline Agent	Evolved Agent	Độ chênh lệch (Δ)
Điểm số F1 trung bình	0.0111	0.1611	+0.1500
Số ca chẩn đoán thành công	6 / 30	8 / 30	+2
Số bước chẩn đoán trung bình	12.33	11.63	-0.70
Số lần gọi công cụ trung bình	11.66	10.90	-0.76
Số lượng token trung bình / ca	64468.17	80771.63	+16303.46
Số công cụ tiến hóa được tạo	0	4	+4

Kết quả bảng cho thấy Evolved Agent đạt điểm số F1 chẩn đoán trung bình cao gấp hơn 14 lần so với Baseline Agent (từ 0.0111 lên 0.1611), đồng thời tăng thêm 2 ca giải quyết thành công hoàn toàn. Đáng chú ý là số lượng bước chẩn đoán và số lần gọi công cụ trung bình đều giảm đi, cho thấy agent sử dụng các công cụ tiến hóa có tính định hướng và chính xác cao hơn, tránh gọi các lệnh rác. Lượng token tiêu thụ trung bình tăng nhẹ do hệ thống phải tải thêm các hướng dẫn mô tả Mastery từ thư viện.

Bảng 4.2 so sánh chi tiết hiệu năng theo từng phân đoạn dữ liệu (Split), giúp đánh giá khả năng học và chuyển đổi tri thức của agent.

Phân tích theo phân đoạn:

- Trên phân đoạn `evolution`, Evolved Agent cải thiện vượt bậc với điểm F1 tăng từ 0.0333 lên 0.5500 và số ca thành công tăng gấp đôi. Điều này chứng minh vòng lặp tự tiến hóa tại test-time hoạt động cực kỳ hiệu quả khi agent vừa điều tra vừa cập nhật công cụ và bộ nhớ.
- Trên phân đoạn `development`, điểm F1 tăng từ -0.1667 lên 0.0000 và giải quyết

Bảng 4.2: Đánh giá hiệu năng theo từng phân đoạn (Split)

Phân đoạn (Split)	Baseline Agent		Evolved Agent		Độ chênh lệch (Δ)	
	F1 Score	Success	F1 Score	Success	F1 Score	Success
evolution	0.0333	2	0.5500	4	+0.5167	+2
development	-0.1667	0	0.0000	1	+0.1667	+1
transfer	0.1667	4	-0.0667	3	-0.2333	-1

thành công thêm 1 ca, chứng minh tri thức đã học được lưu trữ tốt và có thể áp dụng lại để cải thiện chẩn đoán.

- Tuy nhiên, trên phân đoạn `transfer`, hiệu năng của Evolved Agent bị suy giảm nhẹ (F1 giảm 0.2333 và bớt đi 1 ca thành công). Điều này chỉ ra một thách thức quan trọng: hiện tượng quá khớp (overfitting) của các công cụ hoặc quy trình tiến hóa khi áp dụng sang một lớp mạng hoặc kịch bản lỗi hoàn toàn mới mà agent chưa được huấn luyện.

Bảng 4.3 trình bày chi tiết F1 score theo từng nhóm lỗi (Stream) của mạng truyền thông.

Bảng 4.3: Đánh giá hiệu năng theo nhóm lỗi (Stream)

Nhóm lỗi (Stream)	Baseline Agent (F1)	Evolved Agent (F1)	Độ chênh lệch (Δ)
<code>dns_dhcp_service</code>	-0.2222	0.2778	+0.5000
<code>host_ip_config</code>	-0.6111	-0.1667	+0.4444
<code>link_path</code>	0.0556	-0.2222	-0.2778
<code>routing_control</code>	0.3333	0.2778	-0.0555
<code>sdn_p4_control</code>	0.5000	0.6389	+0.1389

4.4 Phân tích chi tiết và thảo luận

4.4.1 Các sự cố có sự cải thiện lớn nhất (Top Gains)

- ****host_missing_ip (evolution split, $\Delta = +2.0$):**** Baseline chẩn đoán sai hoàn toàn do thiết bị thiếu cấu hình IP dẫn đến không ping được gateway. Evolved Agent tự Master mô tả công cụ `get_host_net_config`, truyền đúng tham số thiết bị và xác định được cổng giao tiếp bị thiếu địa chỉ IP.

- ****host_incorrect_gateway** (development split, $\Delta = +2.0$):** Evolved Agent khai thác lại tài liệu Mastery đã lưu từ các tập trước để cấu hình tham số gateway chính xác cho lệnh chẩn đoán, tránh lặp lại lỗi truyền sai tham số của baseline.
- ****bgp_missing_route_advertisement** (evolution split, $\Delta = +2.0$):** Việc chẩn đoán thiếu quảng bá định tuyến yêu cầu kiểm tra cấu hình chi tiết của BGP. Evolved Agent tự động tổng hợp quy trình composite gọi `frr_get_bgp_conf` để kiểm tra trạng thái bảng định tuyến BGP từ xa và tìm ra nguyên nhân chính xác.

4.4.2 Các sự cố bị suy giảm hiệu năng (Top Losses)

- ****ospf_area_misconfiguration** (transfer split, $\Delta = -2.0$):** Sự cố này nằm trong nhóm chuyển giao tri thức. Agent cố gắng gọi một công cụ composite chẩn đoán định tuyến vốn được tối ưu cho mô hình mạng BGP trước đó. Sự khác biệt về định dạng cấu hình tham số của OSPF làm cho các bước gọi công cụ bị lỗi và dẫn đến kết luận sai lệch.
- ****link_flap** (development split, $\Delta = -1.66$):** Lỗi nhấp nhấp chần của link yêu cầu theo dõi trạng thái theo thời gian thực. Việc agent nén vết chẩn đoán quá mức (trajectory compaction) vô tình lọc bỏ các log nhấp chần định kỳ, làm agent mất đi bằng chứng quan trọng để xác định lỗi link_flap.

Chương 5

Kết luận và hướng phát triển

Chương này tổng kết lại những kết quả đạt được của dự án nghiên cứu và phát triển hệ thống Self-Evolving AI Agents trên nền tảng NIKA, đồng thời thảo luận thẳng thắn về những hạn chế hiện tại và đề xuất định hướng phát triển trong tương lai.

5.1 Kết quả đạt được của dự án

Sau quá trình nghiên cứu và thực nghiệm nghiêm túc, dự án đã đạt được một số kết quả quan trọng sau:

- ****Nghiên cứu lý thuyết sâu rộng:**** Tổng hợp và hệ thống hóa các nghiên cứu hàng đầu về AI Agent tự tiến hóa bao gồm cả hướng tiến hóa bộ nhớ (MemInsight, LightMem, A-Mem) và tiến hóa công cụ (DRAFT, TTE, Alita).
- ****Xây dựng kiến trúc hệ thống hoàn chỉnh:**** Tích hợp thành công AI Agent với môi trường giả lập mạng Kathará thông qua giao thức Model Context Protocol (MCP) chuẩn hóa. Hệ thống vận hành tốt trên 3 luồng chẩn đoán phổ biến (ReAct, Plan & Execute, Reflexion).
- ****Hiện thực hóa thành công các mô-đun tiến hóa:****
 1. Mô-đun Bộ nhớ quy trình tự tiến hóa (Composable Procedural Memory Module) sử dụng cấu trúc lai PostgreSQL/Qdrant hỗ trợ nén vết chẩn đoán và cập nhật ngoại tuyến, đặc biệt là cơ chế score-gated validation để tránh rò rỉ đáp án của benchmark.
 2. Mô-đun Tiến hóa công cụ (Tool Evolution Module) cho phép tự Mastery tài liệu hướng dẫn và sinh công cụ mới (composite workflows và generated

Python helpers) có cơ chế kiểm tra AST và thực thi Docker sandbox an toàn.

- **Đánh giá thực nghiệm chi tiết:** Kiểm chứng hệ thống trên 30 incident của benchmark NIKA. Kết quả cho thấy Evolved Agent đạt điểm F1 trung bình vượt trội so với baseline (tăng từ 0.0111 lên 0.1611) và giải quyết thành công thêm nhiều sự cố phức tạp liên quan đến mất cấu hình IP và lỗi định tuyến BGP.

5.2 Những hạn chế hiện tại

Mặc dù đạt được những kết quả khả quan, hệ thống vẫn tồn tại một số hạn chế cần khắc phục:

- **Hiện tượng quá khớp (Overfitting) trên phân đoạn Transfer:** Các công cụ và bài học được tiến hóa trên kịch bản mạng cũ đôi khi bị quá khớp với cấu trúc hoặc giao thức đó. Khi chuyển giao sang phân đoạn `transfer` với các lỗi định tuyến động mới (OSPF), agent dễ gặp lỗi gọi công cụ do không tương thích tham số, làm giảm F1 score.
- **Tăng chi phí Token:** Việc tải thêm các mô tả Mastery dài và các gợi ý bài học từ bộ nhớ vào ngữ cảnh prompt làm tăng lượng token trung bình tiêu thụ mỗi ca từ khoảng 64,468 lên 80,771 tokens.
- **Rủi ro an toàn thực thi ở test-time:** Mặc dù đã có cơ chế kiểm tra cú pháp AST và thực thi Docker sandbox cô lập hoàn toàn mạng để chạy thử nghiệm các công cụ Python mới sinh, việc cho phép LLM tự động sinh mã nguồn thực thi thời gian chạy vẫn tiềm ẩn rủi ro bảo mật lớn nếu đưa vào hệ thống mạng viễn thông thực tế.

5.3 Định hướng phát triển tương lai

Từ những kết quả và hạn chế trên, dự án định hướng một số nội dung nghiên cứu tiếp theo bao gồm:

- **Cải thiện khả năng tổng quát hóa (Generalization):** Nghiên cứu các cơ chế trừu tượng hóa tham số mạnh mẽ hơn trong lúc chẩn đoán và distill công cụ, giúp các công cụ composite workflow dễ dàng thích ứng với các cấu trúc mạng và giao thức khác nhau (giải quyết bài toán Transfer).
- **Tích hợp cơ chế Human-in-the-loop:** Bổ sung giao diện cho phép các kỹ sư mạng con người tham gia phản hồi, đánh giá và phê duyệt (approve) các công

cụ/quy trình mới sinh ra cũng như các ghi nhớ quan trọng trước khi ghi nhận chính thức vào state.json.

- ****Tối ưu hóa hiệu năng và chi phí:**** Tinh chỉnh các mô hình ngôn ngữ lớn chuyên biệt cho miền mạng (Domain-specific LLMs) để giảm kích thước prompt chuẩn đoán và tối ưu hóa thời gian thực thi các Kathará labs.
- ****Mở rộng kịch bản sự cố:**** Đánh giá agent tự tiến hóa trên các kịch bản mạng có quy mô lớn hơn (m, l) và các sự cố phức tạp hơn (compose nhiều lỗi trong một incident).